

6.2 List basics ^

Settings

Beta features

container

A container is a construct used to group related values together and other objects instead of data.

list

A list is a container created by surrounding a sequence of variable `].`

element

A list item is called an element.

index

Elements are ordered by position in the list, known as the element

PARTICIPATION ACTIVITY

6.2.1: Creating lists.

Start ☐ 2x speed

```
>>> prices = ["$20", 14.99, 5]
>>> |
```

Python (command line)

Python interpreter

```
prices = ["$20",
```

prices
"\$20"
14.99
5

Captions ^

1. User creates a new list.
2. The interpreter creates a new object for each list element.
3. 'prices' holds references to objects in the list.

Accessing list elements.

```
→ # Some of the most expensive cars in the world
rolls_royce_la_rose_noire = 30000000 # $30 million
bugatti_la_voiture = 13400000 # $13.4 million
pagani_codalunga = 7400000 # $7.4 million

prices = [
```

PARTICIPATION ACTIVITY

6.2.5: Updating elements in a list.

Start ☐ 2x speed

```
prices = [1.50, 3.75, 6.00]
print("Cost:", prices[0] + prices[1])

# Update the first price
prices[0] = 2.50

# Print the updated cost
print("Updated cost:", prices[0] + prices[1])
```

prices	
2.50	3.75

```
Cost: 5.25
Updated cost: 6.25
```

Captions ^

1. The sum of the first two elements of prices are added together and printed to output.
2. The first element of prices is updated to a value of 2.50 using brackets notation and the f
3. Recomputing the cost of the first two elements gives an updated result.

method

A method instructs an object to perform some action and is specified by the object name followed by a "." symbol and then the method name.

append()

The `append()` list method is used to add new elements to a list.

pop() / remove()

Elements can be removed from a list using the `pop()` or `remove()` methods.

**PARTICIPATION
ACTIVITY**

6.2.7: Adding and removing list elements.

Start

2x speed

```
my_list = [10, "bw"]  
print(my_list)  
  
my_list.append("abc")  
print(f"After append: {my_list}")  
  
my_list.pop(1)  
print(f"After pop: {my_list}")  
  
my_list.remove("abc")  
print(f"After remove: {my_list}")
```

my_list:

10

```
[10, "bw"]  
After append: [10, "bw", "abc"]  
After pop: [10, "abc"]  
After remove: [10]
```

Captions ^

1. `append()` adds an element to the end of the list.
2. `pop()` removes the element at the given index from the list. "bw", which is at index 1, is removed.
3. `remove()` removes the first element with a given value. "abc" is removed and now the list contains only the element 10.

Sequence-type methods

Sequence-type methods are functions built into the Python language for sequence types like lists and strings.

Sequence-type functions

Sequence-type functions are functions built into the Python language like lists or strings as arguments and operate on the given sequence.

Table 6.2.1: Some of the functions and methods useful to lists.

Operation	Description
<code>len(list)</code>	Find the length of the list.
<code>list1 + list2</code>	Produce a new list by concatenating list2 to the end of list1.
<code>min(list)</code>	Find the element in the list with the smallest value. All elements must be of the same type.
<code>max(list)</code>	Find the element in the list with the largest value. All elements must be of the same type.
<code>sum(list)</code>	Find the sum of all elements of a list (numbers only).
<code>list.index(val)</code>	Find the index of the first element in the list whose value matches val.
<code>list.count(val)</code>	Count the number of occurrences of the value val in the list.

Sequence methods and functions.

```
# Concatenating lists
→ house_prices = [380000, 900000, 875000] + [225000]
print(f"There are {len(house_prices)} prices in the list.")

# Find lowest cost housing
cost = min(house_prices)
```

Try 6.2.1: Student grade statistics.

The following program calculates some information regarding final and midterm scores. Try enhancing the range of scores using the functions `min()` and `max()` and by calculating the average midterm and final scores using `sum()` and `len()`.



▶ Run

```
1  # Program to calculate statistics from student test scores.
2  midterm_scores = [99.5, 78.25, 76, 58.5, 100, 87.5, 91, 68, 100]
3  final_scores = [55, 62, 100, 98.75, 80, 76.5, 85.25]
4
5  # Combine the scores into a single list
6  all_scores = midterm_scores + final_scores
7
8  num_midterm_scores = len(midterm_scores)
9  num_final_scores = len(final_scores)
10
11 print(f"{num_midterm_scores} students took the midterm.")
12 print(f"{num_final_scores} students took the final.")
13
14 # Calculate the number of students who took the midterm but not the final
15 dropped_students = num_midterm_scores - num_final_scores
16 print(f"{dropped_students} students must have dropped the class.")
17
18 # Your code goes here
```

CONSOLE



→

**Model Solution** ▼**CHALLENGE
ACTIVITY**

6.2.1: List functions and methods.

763650.1722066.qx3zqy7

Start

```
user_ages = [ 2, 5, 4 ]
```

What is the value of `len(user_ages)`?

1

2

3

4

Check**Next**

View solution ▼ *(Instructors only)*

**CHALLENGE
ACTIVITY**

6.2.2: List basics.

763650.1722066.qx3zqy7

Start

Two ages are read from input into variables age1 and age2. Then, two names are read from input into name1 and name2. Create a list named ages_list to hold age1 and age2, and create a list named names_list to hold name1 and name2.

[Click here to show example](#) ▼

```
1 age1 = int(input())
2 age2 = int(input())
3 name1 = input()
4 name2 = input()
5
6 """ Your code goes here """
7
8 print(ages_list)
9 print(names_list)
```

1

2

3

Check**Next level**

View solution ▼ *(Instructors only)*